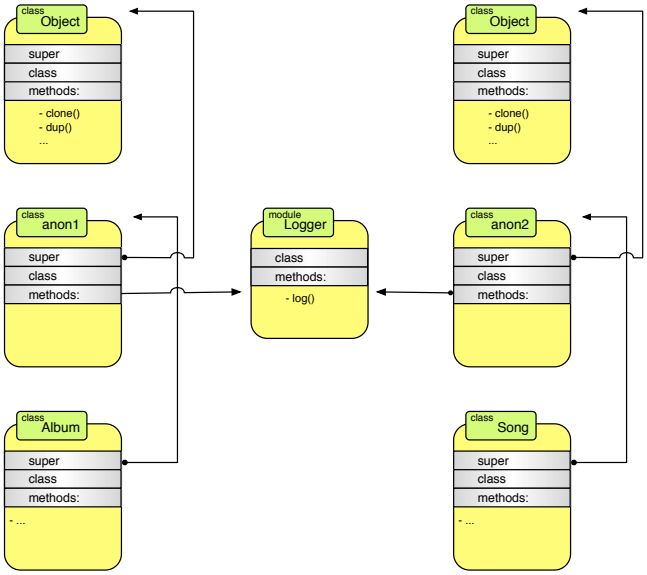


A nice side effect of this arrangement is that if you change a module after including it in a class, those changes are reflected in the class (and the class's objects). In this way, modules behave just like classes:

```
module Mod
  def greeting
    "Hello"
  end
end

class Example
  include Mod
end

ex = Example.new
puts "Before change, greeting is #{ex.greeting}"

module Mod
  def greeting
    "Hi"
  end
end

puts "After change, greeting is #{ex.greeting}"
```

produces:

```
Before change, greeting is Hello
After change, greeting is Hi
```

If a module itself includes other modules, a chain of proxy classes will be added to any class that includes that module, one proxy for each module that is directly or indirectly included.

Finally, Ruby will include a particular module only once in an inheritance chain—including a module that is already included by one of your superclasses has no effect.